

CS3810 Mini-Project 1 - Search Algorithms for a Cleaning Robot

Project Report

Naina Kalra

October 2, 2026

1. Design Decisions

How I Represented States-

For representing states, my main goal was to keep representation as simple and clean as possible so that it works smoothly across DFS, A*, and IDA*. I defined each state as a simple Python tuple: `state = (position, dirty_set)`

- Robot Position is stored as a simple (row, col) tuple.
- Dirty Cells are stored using a frozenset.

Using a frozenset was necessary because standard python sets are mutable, which can cause a `TypeError` when we try to add them to an explored set or use them as keys in a g dictionary. Also, a frozenset is immutable and hashable, so it solved this issue completely.

Environment Rules -

The environment supports five actions which are `MOVE_UP`, `MOVE_DOWN`, `MOVE_LEFT`, `MOVE_RIGHT`, and `CLEAN`. In the `get_actions` method, I ensured that the robot remains within the grid boundaries ($0 \leq \text{row} < \text{rows}$ and $0 \leq \text{col} < \text{cols}$) and never steps onto obstacle cells ('#'). Also, the `CLEAN` action can only be used when the robot is standing directly on a dirty cell. In the state transition function `result()`, I made sure that the original input state is never modified. When performing movement actions, the function returns a new position tuple while keeping the dirty set unchanged. For cleaning actions, it returns a new frozenset created using set subtraction (`dirty_set - {pos}`). Every valid action, whether moving or cleaning has a step cost of 1.

Search Implementations -

For Depth-First Search (DFS), I used a standard LIFO stack with an explored set so that I don't stuck in infinite loops or the robot moving never stops. I reversed the action list (`reversed(actions_list)`) before adding actions to the stack so that `MOVE_UP` gets popped first, to make sure the search is predictable and deterministic. The main difference between DFS and A* is that A* uses a priority queue to pick the state with the lowest total cost ($f(n) = g(n) + h(n)$). I stored the best-known g-value for each state in a dictionary. If a state was found again through a cheaper path (`new_cost < g[successor]`), its cost was updated, and it was pushed back into the priority queue. Iterative Deepening A* (IDA*) runs repeated depth-first searches bounded by a f-

cost limit. When a node exceeds the threshold, the smallest exceeded value becomes the new threshold for the next pass. In the experiments, IDA* expanded a very large number of nodes on bigger grids, especially when using weaker heuristics. For all algorithms, I recorded performance metrics including expanded nodes, frontier size, and iterations.

2. Heuristic Analysis

h0

h0 always returns 0, so it gives no useful information about the work left.

h1

h1 counts how many dirty cells are left. Every dirty cell needs at least one CLEAN action, and every action costs 1. So, if there are k dirty cells left, the real remaining cost has to be at least k. h1 is admissible because it just returns k and it never guesses a cost higher than the true cost.

h2

h2 adds the number of remaining dirty cells and the Manhattan distance from the robot to the closest dirty cell together. The dirty cells count is a safe lower estimate for the same reason as h1. On top of that, the robot must walk to at least one dirty cell before cleaning, and Manhattan distance gives the shortest possible path to reach the nearest one on an open grid. Obstacles can only make the path longer, never shorter, so the real number of steps is always equal to or greater than the Manhattan distance. Since both parts are safe lower estimates, adding them together never overestimates the true cost, so h2 is also admissible.

Heuristic Comparison

The test results show a clear pattern when comparing the three heuristics. Across all six test grids, A* with h2 expanded fewer nodes than h1 and h1 expanded fewer nodes than h0. For example, on grid 6, the number of expanded nodes dropped from 10,873 with h0, to 7,665 with h1 and down to 5,797 with h2. This shows that a better heuristic gives A* a smarter estimate of the remaining cost, helping it avoid unhelpful paths and reducing the total number of nodes expanded.

3. Experimental Results

Results Table

Table is given below for experimental results recorded across all six test grids –

grid	algorithm	heuristic	status	cost	nodes_expanded	max_frontier	iterations	seconds
g1_tiny	astar	h0	ok	7.0	21.0	4.0		0.0001
g1_tiny	astar	h1	ok	7.0	20.0	5.0		0.0001
g1_tiny	astar	h2	ok	7.0	13.0	4.0		0.0001
g1_tiny	astar	h3	SKIP					
g1_tiny	dfs	-	ok	12.0	16.0	5.0		0.0
g1_tiny	idastar	h0	ok	7.0	95.0		8.0	0.0001
g1_tiny	idastar	h1	ok	7.0	52.0		6.0	0.0001
g1_tiny	idastar	h2	ok	7.0	27.0		4.0	0.0001
g1_tiny	idastar	h3	SKIP					
g2_open	astar	h0	ok	12.0	98.0	17.0		0.0002
g2_open	astar	h1	ok	12.0	96.0	18.0		0.0002
g2_open	astar	h2	ok	12.0	76.0	21.0		0.0002
g2_open	astar	h3	SKIP					
g2_open	dfs	-	ok	36.0	48.0	32.0		0.0001
g2_open	idastar	h0	ok	12.0	15586.0		13.0	0.0289
g2_open	idastar	h1	ok	12.0	3821.0		10.0	0.0072
g2_open	idastar	h2	ok	12.0	616.0		4.0	0.0016

g2_open	idastar	h3	SKIP					
g3_blocks	astar	h0	ok	15.0	411.0	61.0		0.001
g3_blocks	astar	h1	ok	15.0	331.0	71.0		0.0008
g3_blocks	astar	h2	ok	15.0	256.0	87.0		0.0008
g3_blocks	astar	h3	SKIP					
g3_blocks	dfs	-	ok	43.0	69.0	34.0		0.0001
g3_blocks	idastar	h0	ok	15.0	29281.0		16.0	0.0551
g3_blocks	idastar	h1	ok	15.0	4463.0		11.0	0.0083
g3_blocks	idastar	h2	ok	15.0	2010.0		8.0	0.0056
g3_blocks	idastar	h3	SKIP					
g4_pillars	astar	h0	ok	19.0	1160.0	161.0		0.0026
g4_pillars	astar	h1	ok	19.0	850.0	201.0		0.0019
g4_pillars	astar	h2	ok	19.0	510.0	191.0		0.0018
g4_pillars	astar	h3	SKIP					
g4_pillars	dfs	-	ok	80.0	110.0	50.0		0.0003

g4_pillars	idastar	h0	ok	19.0	58097.0		20.0	0.1067
g4_pillars	idastar	h1	ok	19.0	8883.0		14.0	0.0158
g4_pillars	idastar	h2	ok	19.0	4174.0		12.0	0.0109
g4_pillars	idastar	h3	SKIP					
g5_rooms	astar	h0	ok	25.0	2403.0	237.0		0.0064
g5_rooms	astar	h1	ok	25.0	2097.0	274.0		0.0064
g5_rooms	astar	h2	ok	25.0	1651.0	380.0		0.0061
g5_rooms	astar	h3	SKIP					
g5_rooms	dfs	-	ok	92.0	136.0	68.0		0.0002
g5_rooms	idastar	h0	ok	25.0	17817953.0		26.0	38.994 2
g5_rooms	idastar	h1	ok	25.0	1802996.0		19.0	3.8046
g5_rooms	idastar	h2	ok	25.0	731040.0		17.0	2.2225
g5_rooms	idastar	h3	SKIP					
g6_corridor	astar	h0	ok	32.0	10873.0	940.0		0.0278
g6_corridor	astar	h1	ok	32.0	7665.0	1066.0		0.019
g6_corridor	astar	h2	ok	32.0	5797.0	1282.0		0.019
g6_corridor	astar	h3	SKIP					

g6_corridor	dfs	-	ok	121.0	195.0	90.0		0.0005
g6_corridor	idastar	h0	TIMEOUT					60.0044
g6_corridor	idastar	h1	ok	32.0	22802235.0		24.0	52.1129
g6_corridor	idastar	h2	ok	32.0	9126492.0		21.0	30.6362
g6_corridor	idastar	h3	SKIP					

Figures and Captions

Figure 1: Scaling Comparison

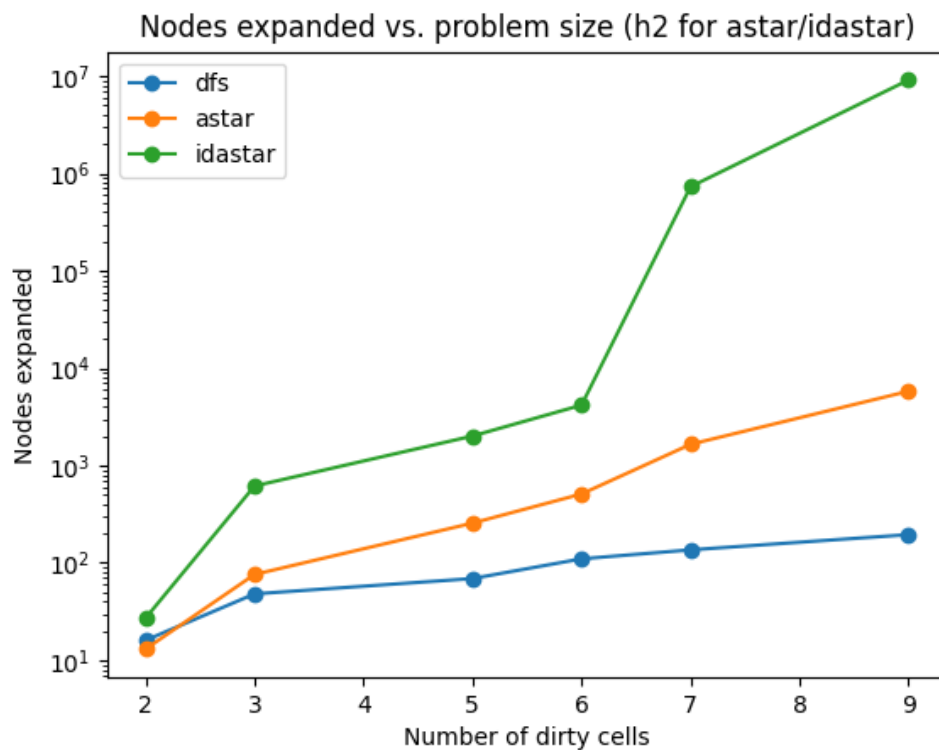


Figure 1: *Nodes expanded vs. problem size using h2 for A and IDA**.

As the number of dirty cells increases from 2 to 9, the search space grows significantly. DFS expands fewer nodes by taking a direct non-optimal path, IDA* expands exponentially more nodes on larger grids compared to A* because it repeatedly re-visits states across iterations.

Figure 2: Heuristic Impact Comparison

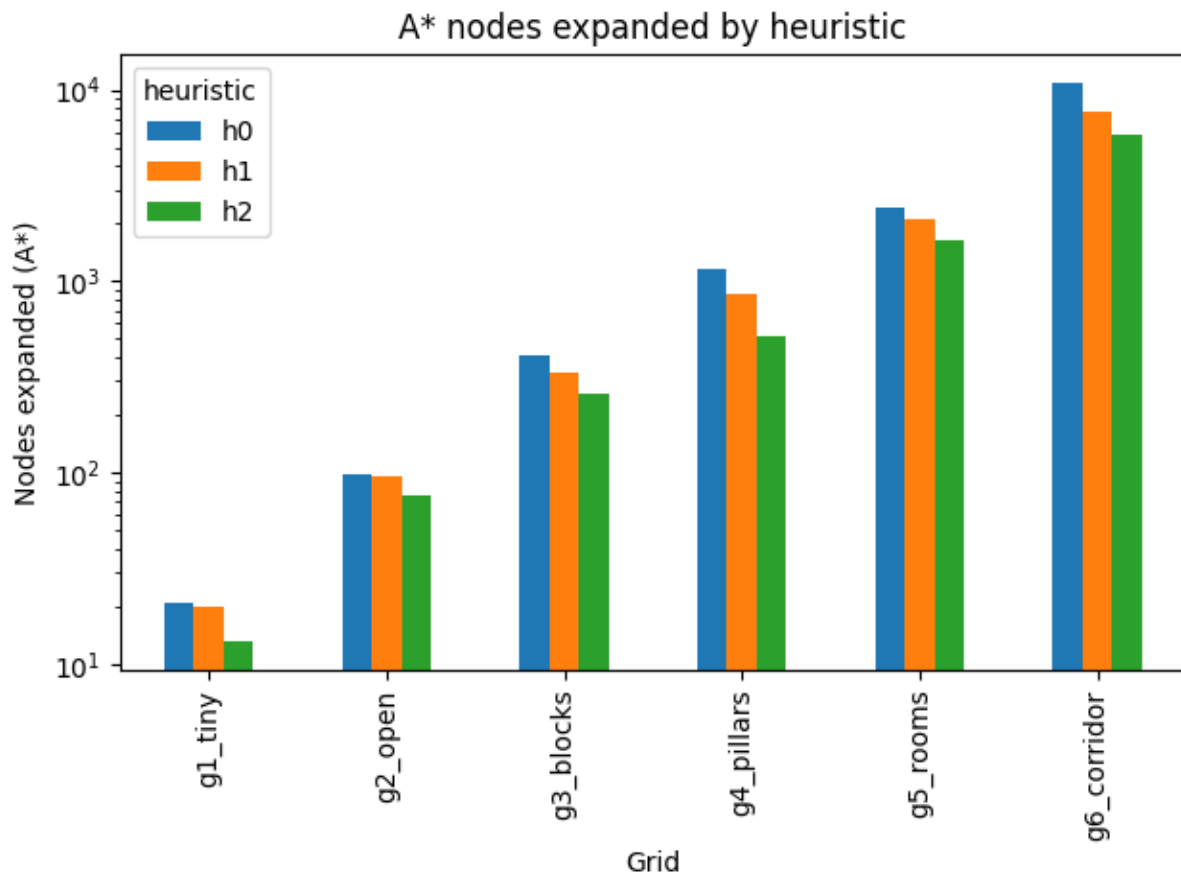


Figure 2: Comparison of nodes expanded by A using different heuristics (h0, h1, h2) across six grid configurations.

Across all grids (from g1_tiny to g6_corridor), h2 consistently expands the fewest nodes compared to h1 and h0. This proves that a more informative heuristic provides better guidance and reduces unnecessary search exploration.

4. Discussion -

1. Why can DFS expand fewer nodes but produce a worse solution?

DFS just stops as soon as it finds any path to the goal even without checking if a shorter path exists. That's why it expanded only 195 nodes on g6_corridor but gave a high cost of 121. While, A* expands states using $f(n) = g(n) + h(n)$, which means it keeps searching until it is sure it found the cheapest path. This costs more node expansions (5,797 nodes) but gives the optimal path cost of 32.

2. How do h0, h1, and h2 affect A*?

Comparing the heuristics shows a clear pattern. Across all six test grids, A* with h2 expanded fewer nodes than h1, and h1 expanded fewer than h0. For example, on g6_corridor, the number of expanded nodes dropped from 10,873 with h0, to 7,665 with h1, and down to 5,797 with h2. A more informative heuristic gives A* a better estimate of the remaining cost, helping it avoid unhelpful paths and reducing the total number of expanded nodes.

3. Why might IDA* be useful despite expanding more nodes?

Even though IDA* expands many more nodes than A*, it is still useful because it uses much less memory. IDA* does a depth-first search instead of keeping a large priority queue in memory like A*. The downside is it repeats a lot of work across iterations. For example, on g6_corridor, IDA* with h2 expanded 9,126,492 nodes, while A* with h2 needed only 5,797 nodes. Still, IDA* with h2 finished in about 30.6 seconds, whereas IDA* with h0 timed out completely. IDA* is a great choice when memory is limited, as long as it uses a strong heuristic.

4. State-Space Size

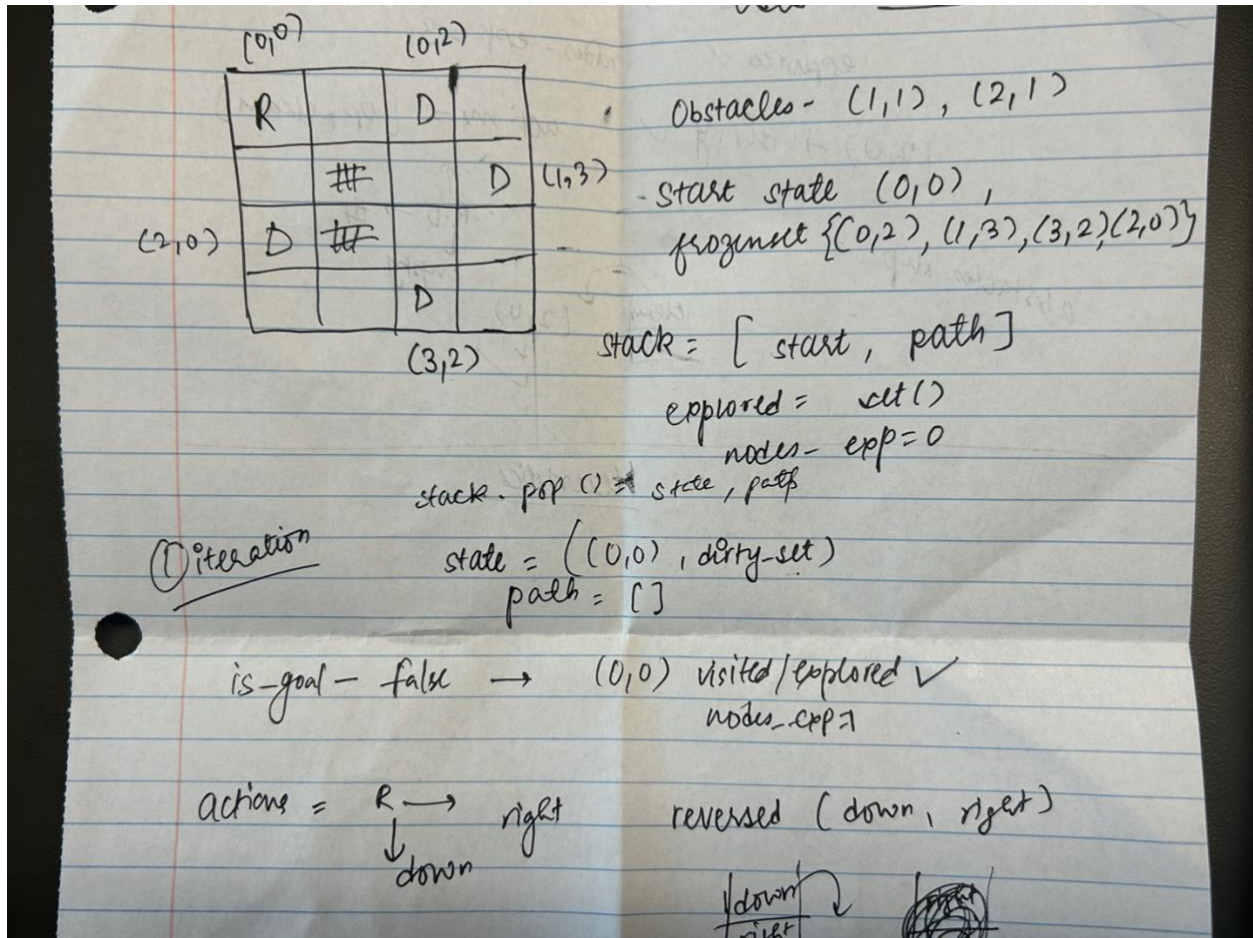
For a grid with R rows, C columns, and k dirty cells, the maximum number of possible states is $(R \times C) \times 2^k$. The robot can be in R X C possible positions, and each of the k dirty cells can either be dirty or cleaned (2^k combinations). Multiplying them gives the total state space upper bound. In practice, the actual reachable states are fewer because obstacles block some spots. This formula shows why the search space grows exponentially as the number of dirty cells (k) increases.

5. Which grid was hardest and why?

The g6_corridor grid was the hardest grid in the experiments, especially for IDA* because with the weak h0 heuristic, IDA* timed out completely. With h1, it expanded 22,802,235 nodes, and even with the best heuristic h2, it needed 9,126,492 nodes and 30.6 seconds to finish as compared to only 5,797 nodes for A*. The narrow corridor limits the robot's movement options, forcing the algorithm to explore many dead ends. This narrow layout in my opinion made g6_corridor much harder than the other grids.

6. Roomba Assumption

My program assumes that the robot already knows the full grid layout and obstacle positions before starting. If the robot does not know the map in advance, it could not plan a complete route before. Instead, it would have to explore while moving, update its map as it sees new obstacles, and re-plan its route on the way. Instead, it must use an online search approach, where it moves step-by-step, uses its sensors to see new dirt or obstacles, and continually updates its plan as it goes.



Reflection -

What broke in implementation -

Building this project involved fixing several critical bugs in different parts of the codebase. In the very start, a trailing comma in the class initialization (`self.grid = grid,`) converted the grid matrix into a tuple, which caused unexpected type errors and made me think the code seems correct, but where is the error coming from. In the environment model, the goal check function `is_goal()`, I missed the `return False` in its else branch and it returned `None` and broke search loop

conditions. I also made a variable naming mistake in `get_actions()`, writing `action.append(action)` instead of `actions.append(action)`, which gave an `AttributeError` on string objects. Then there was this error in `result()` in transition model that initially only handled the CLEAN action and it was causing invalid state transitions whenever the robot moved. I had some indentation errors in DFS also allowed loop logic to execute outside the main condition, causing unguided iterations. Also, in A*, checking only whether a state was unvisited was stopping updating paths when a cheaper route was found, requiring me to add the proper `new_cost < g[successor]` check. In IDA*, managing both the state path for cycle detection and the action path for the solution during recursive backtracking was tricky to keep in sync for me.

How I Found and Fixed These Issues -

I used a combination of step-by-step debugging strategies and AI to fix these bugs. I added temporary print statements to inspect the active path stack to see where exactly the state synchronization was failing during backtracking in IDA*. For the errors I got like in indentation of code or using something wrong, I used AI to understand where it is going wrong, as it appeared correct to me. Before running full graph searches, I tested individual functions like `is_goal()`, `get_actions()`, and `result()` using small mock inputs. This unit testing approach helped me to see the missing return value in `is_goal()` and the missing movement actions in `result()`. To fix the subtle tuple bug caused by the trailing comma, I used AI as I was feeling its correct on my end.

What I Would Do Differently

If I had to do this project again, I would change a few simple things to save time and avoid silly mistakes. First, I would test small helper functions like `is_goal()` and `result()` with simple inputs before writing the main search algorithms. I also spent a lot of time manually running each grid with different algorithms and copying numbers into Excel, so next time I would write a simple loop script to run all tests and save the results automatically. Also, I would pay more attention to variable names and indentation from day one to avoid time-wasting bugs like `action.append` instead of `actions.append`. And I would try using simple testing frameworks or built-in tools like `pytest` to automatically test my functions, which would make debugging much easier and faster.